# DNSGuard-X

Project report and end-to-end workflow

**An explainable, rule-based monitor for suspicious DNS lookups in saved captures or live traffic.**

**What it is.** DNSGuard-X is a Python prototype that identifies patterns associated with DNS tunneling, domain-generation algorithms (DGA), high-rate DNS floods, amplification, and possible cache poisoning. It reports observations with a severity, host, domain, and human-readable reason. It does not block traffic or prove that an attack occurred.

**Current status.** All principal components described in the repository handout and workflow are implemented in source. The linked full project report was missing and is supplied by this document. Runtime correctness has not been signed off in this checkout: the local Python lacks core dependencies, PyPI was unreachable, and live capture has not been exercised. See **docs/PROJECT_STATUS.md** for the audit and remaining checks.

## 1. Architecture at a glance

| Step | Input and decision | Implementation |
|---|---|---|
| 1. Capture | Read a PCAP/PCAPNG, or sniff DNS on UDP/TCP port 53. | capture.py |
| 2. Parse | Decode packet into a normalised DNSRecord; ignore non-DNS. | parser.py |
| 3. Preprocess | Normalise name; derive registered parent and subdomain. | features.py |
| 4. Whitelist | Skip known-safe parent domains but retain matching state. | whitelist.py / engine.py |
| 5. Features | Measure one query and 60-second behaviour per source host. | features.py |
| 6. Detect | Run D1-D5 on queries; D4/CP also inspect responses. | detectors.py / engine.py |
| 7. Alert | Merge repeated findings into explainable, counted alerts. | alerts.py |
| 8. Persist / show | SQLite, CLI, Streamlit dashboard, React/FastAPI UI. | storage.py / main.py / web/ |

**The orchestration point:** `DetectionEngine.process_record()` in `dnsguard/engine.py`. Both offline replay and live sniffing call this same method; the dashboards consume its output or the SQLite run data.

## 2. Packet and feature model

**Capture and parse.** Scapy streams packets from disk in order, or sniffs a chosen interface with a DNS BPF filter. The parser records timestamp, source/destination IP, lowercased question name, query type, response flag and code, transaction ID, packet size, answer records, minimum TTL, and UDP/TCP transport. A sorted host pair and DNS ID help associate a reply with a request.

**Parent domain.** An offline public-suffix snapshot identifies the registered domain: `a.x.evil.com` and `b.x.evil.com` group as `evil.com`. The left-hand labels are analysed for encoded or algorithmic names. The whitelist stores parent domains, so a listed parent suppresses its subdomains. Requests still enter the response matcher to avoid orphaning a later reply.

**Query-level signals.** Name length, longest label, number of labels, character entropy of the subdomain, digit/vowel ratios, longest consonant run, record type, and (when a response arrives) the reply/request byte ratio. Shannon entropy is `H = -sum(p_i log2 p_i)`; random-looking strings generally score higher than repeated or simple strings.

**Host-level signals.** A deque per source IP keeps a sliding 60-second window. It calculates lookup rate, the largest count of unique subdomains beneath a single parent, fraction of NXDOMAIN responses, response count, and TXT/NULL share. Responses count toward the client host (destination of the reply). The window and minimum rate span are configured in `config.yaml`.

## 3. Rules and default settings

| Rule | Trigger from config.yaml | Meaning / severity |
|---|---|---|
| D1 | Subdomain entropy > 3.8 AND full name length > 50. | Encoded payload in a DNS name; High. |
| D2 | More than 30 unique subdomains of the same parent in 60 s. | Repeated data-bearing lookups; High. |
| D3 | At least 20 responses AND NXDOMAIN fraction > 0.5. | DGA/C2 discovery or water torture; Medium. |
| D4 | Host QPS > 100 OR ANY reply/request bytes > 10x. | Flood or amplification; High. |
| D5 | Subdomain entropy > 3.5 AND vowel share < 0.2 OR consonant run >= 5. | Possible machine-made name; Low. See note below. |
| CP | Conflicting replies with same matched ID, or unsolicited reply after grace. | Possible forged answer; Medium. |

**D5 logic detail:** The actual rule is entropy > 3.5 AND (vowel share < 0.2 OR consonant run >= 5), with a nonempty subdomain. CP is a response-level check in the engine, not a sixth `Detector` subclass. D4 rate runs on queries; its amplification branch uses matched responses. All operational thresholds are exposed in configuration.

Each hit is a Finding with detector ID, type, severity, source host, parent domain, timestamp, and reason. Several rules can fire for one record. The reason describes the measured behaviour, while the UI also provides a simple category explanation.

## 4. From findings to a useful warning

**Grouping.** AlertManager merges findings with the same source IP, domain/group, and attack type when hits occur within the configured 60-second interval. It increments the hit count, updates the last timestamp, and retains the highest severity. D3 is grouped at host level because its NXDOMAIN pattern spans many unrelated domains. A new interval creates another in-memory alert.

**Persistence.** The CLI writes three SQLite tables: `runs` (source and totals), `queries` (per-inspected-lookup statistics), and `alerts` (deduplicated warnings). Alerts are upserted as counts increase; a later de-duplication window keeps its own row. Older databases are migrated to this key. The API's on-demand sample and upload analyses return JSON directly; they do not automatically create a stored run. SQLite runs come from the CLI.

**Displays.** CLI prints each new warning and a summary of categories, top hosts, and severities. Streamlit accepts an uploaded capture or a saved database and draws category/time/entropy charts, top-host summaries, and a filterable table. The React UI calls a FastAPI backend, which serves the committed production bundle on localhost:8000 and exposes sample, upload, health, and stored-run routes.

## 5. Example walkthroughs

### A possible tunneling attempt

A client asks for many long, encoded labels beneath `data-exfil-demo.net`. D1 can flag a sufficiently long, high-entropy name immediately. As different subdomains accumulate in the window, D2 can flag the volume. Separate findings may become two alert categories; repeated hits within each category increase a count instead of filling the UI with thousands of rows.

### A host trying many nonexistent names

The host sends random-looking domain lookups. D5 may flag individual suspicious subdomain names, but it requires a nonempty subdomain. Once at least 20 responses have been observed and more than half are NXDOMAIN, D3 can fire on a later query; its alert is aggregated by host.

### Large answers and conflicting replies

A matched ANY request whose response is over 10 times the request size can trigger D4 amplification. The matcher also remembers an answered transaction briefly. A later reply with a different answer set, or an unmatched response after the start-of-capture grace, produces a CP indicator. Such signals are not proof of successful cache poisoning; DNSSEC is outside this prototype.

## 6. Reproducing and validating

| Action | Command from repository root |
|---|---|
| Install | python -m pip install -r requirements.txt -r web/requirements.txt |
| Make sample data | python tools/generate_traffic.py |
| Analyse PCAP | python -m dnsguard.main --pcap data/pcaps/mixed.pcap |
| Score synthetic data | python tools/evaluate.py |
| Run tests | python -m pytest -q |
| Web UI | python web/server.py (browse localhost:8000) |
| Streamlit | streamlit run dashboard/app.py |

The generator writes normal, tunneling, DGA, flood, amplification, poisoning, and mixed PCAPs plus query labels under `data/pcaps/`; it does not send network packets. The evaluator reports per-category precision/recall/F1 and benign false positives. Its current host/domain-level association can attribute a finding to other queries from the same host/domain in the same run, so the synthetic query-level scores should be treated as demonstration figures, not field validation.

**Verification during this review:** Python source compilation and a focused SQLite storage/migration smoke check passed. Pytest could not collect because the host interpreter lacked scapy and tldextract. Installing requirements failed when PyPI DNS lookup was unavailable. New full-suite regression tests for separate alert windows remain pending. No live interface or Npcap test was run. A fresh frontend build requires npm packages; the committed `web/frontend/dist` supports the documented clone-and-serve path.

## 7. Completion and sensible extensions

**Assessment:** The documented feature paths are present in source; runtime acceptance remains open until dependencies, generated captures, test suite, evaluator, UI endpoints, and relevant live-capture environment are exercised. This report fills the missing docs deliverable. It is an explainable class-project prototype, not a validated production IDS.

**Next:** (1) event-level evaluation on held-out, varied captures; (2) per-network adaptive baselines and whitelist controls; (3) JSON/syslog/SIEM export plus analyst triage; (4) an optional, explainable ML rule plugged into the existing Detector interface. Treat DNS over HTTPS visibility, DNSSEC validation, and active blocking as separate design decisions.

**Source map:** onboarding/AGENTS.md; onboarding/HANDOUT.md; onboarding/workflow.pdf; README.md; config.yaml; dnsguard/{capture,parser,features,whitelist,detectors,engine,alerts,storage,main}.py; tools/{generate_traffic,evaluate}.py; dashboard/app.py; web/server.py; web/frontend/src/. This report describes the checkout reviewed on 21 October 2026.